



CHAPTER 6 — Basic String Programs



Goal

Use the string concepts learned so far to write **small, simple programs**.



SLIDE 1 — Find the Length of a String



What are we finding?

We want to find **how many characters** are in a string.

Example:

"HELLO"

H E L L O

↑ ↑ ↑ ↑ ↑

1 2 3 4 5



Length = 5



Simple Program

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main()
```

```
{
```

```
    char word[] = "HELLO";
```

```
    printf("Length = %d", strlen(word));
```

```
    return 0;
```

```
}
```



What is happening?

word
↓
"HELLO"
↓
strlen()
↓
5

💡 **Remember:**

`strlen()` → counts the characters in a string 📏

🎯 **Try:** Change "HELLO" to "C PROGRAM" and see what happens.

SLIDE 2 — Reverse a String

🤔 **What does Reverse mean?**

Reverse means writing the string **from the last character to the first**.

Before: C O D E

↓ ↓ ↓ ↓

After: E D O C

💻 **Simple Program**

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main()
```

```
{
```

```
    char word[] = "CODE";
```

```
    int i;
```

```
    for(i = strlen(word) - 1; i >= 0; i--)
```

```
    {
```

```
        printf("%c", word[i]);
```

```
}  
  
    return 0;  
}
```

Output:

EDOC

Let's understand the loop

For "CODE":

C	O	D	E
0	1	2	3

↑
Start

The loop goes:

$3 \rightarrow 2 \rightarrow 1 \rightarrow 0$

So we get:

$E \rightarrow D \rightarrow O \rightarrow C$

Memory Trick:

Normal order $\rightarrow 0 \rightarrow \text{last}$

Reverse order $\rightarrow \text{last} \rightarrow 0$ 

SLIDE 3 — Check for Palindrome

What is a Palindrome?

A palindrome is a word that is **the same when read forward and backward**.

Examples:

MADAM → MADAM ✓

LEVEL → LEVEL ✓

HELLO → OLLEH ✗

Think of a Mirror

M A D A M

↑ ↑
M M ✓

↑ ↑
A A ✓

The characters match from both ends.

Beginner Idea

Instead of writing a complicated program, understand the steps first:

- 1 Take a string
↓
- 2 Look at the first and last characters
↓
- 3 Compare them
↓
- 4 Move towards the middle
↓
- 5 If all match → Palindrome ✓

Remember

Same forward + backward = Palindrome 

SLIDE 4 — Count Vowels and Consonants

🤔 What are we doing?

We check each character and decide whether it is a **vowel** or **consonant**.

A Vowels

A E I O U

Example:

APPLE

A → Vowel ●

P → Consonant ●

P → Consonant ●

L → Consonant ●

E → Vowel ●

So:

Vowels = 2

Consonants = 3

💻 Simple Logic

Take one character



Is it A, E, I, O or U?



YES NO



Vowel Consonant



count++ count++

🧠 Memory Trick

Check → Decide → Count 🔍 →

1	2
3	4

💡 For the first version of the program, students can work with **simple alphabetic strings** such as "APPLE".



SLIDE 5 — Compare Two Strings



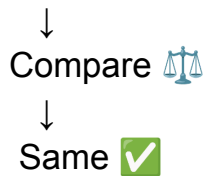
Why compare strings?

Sometimes we need to check whether **two strings are the same**.

Example:

String 1 → "HELLO"

String 2 → "HELLO"



Simple Program

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main()
```

```
{
```

```
    char first[] = "HELLO";
```

```
    char second[] = "HELLO";
```

```
    if(strcmp(first, second) == 0)
```

```
    {
```

```
        printf("Same");
```

```
    }
```

```
    else
```

```
    {
```

```
        printf("Different");
```

```
    }
```

```
    return 0;
```

```
}
```

Output:

Same

 **What is happening?**

`strcmp()` compares the two strings.

"HELLO"  "HELLO"
↓
Same 

 **Remember:**

`strcmp()` → Compare two strings 

CHAPTER 6 RECAP

 **Length** → Find how many characters

 **Reverse** → Read from last to first

 **Palindrome** → Same forward and backward

 **Vowels & Consonants** → Check and count

 **Compare** → Check two strings

 **BIG MEMORY TRICK**

COUNT → REVERSE → CHECK → COUNT → COMPARE

QUESTIONS:-

 **Q1. MCQ — Find String Length**

What will be the output of this program?

```
#include <stdio.h>

#include <string.h>

int main()

{

    char word[] = "CODE";

    printf("%d", strlen(word));

    return 0;

}
```

- ♦ A) 3
- ♦ B) 4
- ♦ C) 5
- ♦ D) 0

 **Hint:** Count the characters in "CODE".

 Glossary:

-  **String length** — The number of characters in a string.
-  **strlen()** — Finds the length of a string.

 **Answer: B) 4**

 Q2. MCQ — Reverse a String

If the string is:

```
char word[] = "CAT";
```

What will be its **reverse**? 

- ♦ A) TAC
- ♦ B) CTA
- ♦ C) ACT
- ♦ D) CAT

 **Hint:** Start reading from the last character.

 **Glossary:**

-  **Reverse** — Arrange the characters in the opposite order.

 **Answer: A) TAC**

Q3. Fill in the Blank — String Length

Complete the code to find the **length** of the string.

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main()
```

```
{
```

```
    char word[] = "HELLO";
```

```
    printf("Length = %d", _____(word));
```

```
    return 0;
```

```
}
```

 **Hint:** Use the function that finds the length of a string.

 **Glossary:**

-  **Length** — The number of characters in the string.
-  **strlen()** — Finds the length of the string.

✓ **Answer:** `strlen`

Q4. Fill in the Blank — Counting Vowels

Complete the code to **count vowels** in the string.

```
char word[] = "APPLE";
```

```
int vowels = 0;
```

```
for(int i = 0; word[i] != '\0'; i++)
```

```
{
```

```
    if(word[i] == 'A' || word[i] == 'E')
```

```
    {
```

```
        _____;
```

```
    }
```

```
}
```

```
printf("%d", vowels);
```



Hint: When a vowel is found, increase the `vowels` count by 1.



Glossary:

-  **Vowel** — One of `A`, `E`, `I`, `O`, or `U`.
-  **Count** — The number of vowels found.

✓ **Answer:** `vowels++`

Q5. Code Choice — Compare Two Strings

Which code correctly **compares** the two strings?

```
char first[] = "CAT";
```

```
char second[] = "CAT";
```

- ♦ **A:** `if(first == second)`
`printf("Same");`
- ♦ **B:** `if(strcmp(first, second) == 0)`
`printf("Same");`
- ♦ **C :** `if(first = second)`
`printf("Same");`
- ♦ **D:** `if(strlen(first) == strlen(second))`
`printf("Same");`

 **Hint:** Use the string function specifically meant to compare strings.

Glossary:

-  **Compare** — Check two strings to see whether they are the same or different.
-  **strcmp()** — Compares two strings.

 **Answer: B)** `strcmp(first, second) == 0`